

Introduction to PyTorch library

PyTorch – analogue of Numpy, where for multidimensional tensors you can do:

- Computations on GPU
- Automatic differentiation with dynamic computational graph
- Efficient memory management

Numpy vs. Pytorch

```
x = np.array([1., 2., 3.])  
y = torch.tensor([1., 2., 3.])
```

```
print(x, y)  
print(x.dtype, y.type())
```

```
[1. 2. 3.] tensor([1., 2., 3.])  
float64 torch.FloatTensor
```

```
x.reshape([1,2,8]) -> x.view(1,2,8)  
x.sum(axis=-1) -> x.sum(dim=-1)  
x.astype('int64') -> x.type(torch.LongTensor)
```

```
torch.from_numpy(npx) -- returns Tensor  
tt.numpy() -- returns Numpy Array
```

```
# numpy world
```

```
x = np.arange(16).reshape(4, 4)
```

```
print("X :\n%s" % x)  
print("add 5 :\n%s" % (x + 5))  
print("X*X^T :\n", np.dot(x, x.T))  
print("mean over cols :\n%s" % (x.mean(axis=-1)))  
print("cumsum of cols :\n%s" % (np.cumsum(x, axis=0)))
```

```
# pytorch world
```

```
x = torch.arange(0, 16).view(4, 4).float()  
print('Type of x:', type(x))
```

```
print("X :\n%s" % x)  
print("add 5 :\n%s" % (x + 5))  
print("X*X^T :\n", torch.matmul(x, x.transpose(1, 0)))  
print("mean over cols :\n", torch.mean(x, dim=-1))  
print("cumsum of cols :\n", torch.cumsum(x, dim=0))
```

Useful operations with tensors

```
x = torch.zeros(2, 1, 2, 1, 2)
print(x.size())
y = x.squeeze()
print(y.size())
z = x.squeeze(1)
print(z.size())
```

```
torch.Size([2, 1, 2, 1, 2])
torch.Size([2, 2, 2])
torch.Size([2, 2, 1, 2])
```

```
x = torch.zeros(2, 2, 2)
print(x.size())
y = x.unsqueeze(1)
print(y.size())
```

```
torch.Size([2, 2, 2])
torch.Size([2, 1, 2, 2])
```

```
x = torch.tensor([[1, 2, 3], [4, 5, 6]])
print(x)
y = torch.cat((x, x, x), 0)
print(y)
z = torch.cat((x, x, x), 1)
print(z)
```

```
tensor([[1, 2, 3],
        [4, 5, 6]])
tensor([[1, 2, 3],
        [4, 5, 6],
        [1, 2, 3],
        [4, 5, 6],
        [1, 2, 3],
        [4, 5, 6]])
tensor([[1, 2, 3, 1, 2, 3, 1, 2, 3],
        [4, 5, 6, 4, 5, 6, 4, 5, 6]])
```

In-place operations

`x.exp()` # not-in-place operation

`x.exp_()` # in-place operation

```
x = torch.arange(4).float()
print('Not-in-place:')
print('\tx.exp():\t\t', x.exp().numpy())
print('\tx:\t\t\t', x.numpy())
print('In-place:')
print('\tx.exp_():\t\t', x.exp_().numpy())
print('\tx after x.exp_():\t', x.numpy())
```

Not-in-place:

<code>x.exp():</code>	<code>[1. 2.7182817 7.389056 20.085537]</code>
<code>x:</code>	<code>[0. 1. 2. 3.]</code>

In-place:

<code>x.exp_():</code>	<code>[1. 2.7182817 7.389056 20.085537]</code>
<code>x after x.exp_():</code>	<code>[1. 2.7182817 7.389056 20.085537]</code>

In-place operations

```
x = torch.arange(0, 4).view(2, 2)
y = torch.arange(4, 8).view(2, 2)
z = torch.arange(8, 12).view(2, 2)

# Not-in-place:
u = x + 2 * y - z      # 3 array allocations
print(u.numpy())

# In-place
u = y.clone()          # 1 array allocation
u.mul_(2)
u.add_(x)
u.sub_(z)
print(u.numpy())
```

Broadcasting

```
a = torch.Tensor([1, 1, 1, 2]).view(4, 1)
b = torch.Tensor([1, 0, 1, 0]).view(1, 4)
c = torch.randn(16).view(4, 4)
print('a:', a)
print('b:', b)
print('a + b:', a + b)
print('c:', c)
print('b + c:', b + c)
```

```
a: tensor([[1.],
          [1.],
          [1.],
          [2.]])
b: tensor([[1., 0., 1., 0.]])
a + b: tensor([[2., 1., 2., 1.],
              [2., 1., 2., 1.],
              [2., 1., 2., 1.],
              [3., 2., 3., 2.]])
c: tensor([[ -0.2284,  1.0384,  1.3887, -0.4445],
          [ 0.3789,  0.1920,  0.5735, -0.3367],
          [-0.9887, -0.7452,  0.4337,  0.8380],
          [ 0.3519,  1.2024,  1.4539, -1.5165]])
b + c: tensor([[ 0.7716,  1.0384,  2.3887, -0.4445],
              [ 1.3789,  0.1920,  1.5735, -0.3367],
              [ 0.0113, -0.7452,  1.4337,  0.8380],
              [ 1.3519,  1.2024,  2.4539, -1.5165]])
```

Computations on GPU

```
device = torch.device('cuda:0' if torch.cuda.is_available() else 'cpu')

x = torch.tensor([1, 2, 3])
print(x.device)
y = x.cuda()
print(y.device)
z = y.cpu()
print(z.device)
u = x.to(device)
print(u.device)
v = torch.zeros(4, device=device)
print(v.device)
```

```
cpu
cuda:0
cpu
cuda:0
cuda:0
```

Automatic differentiation

```
x = torch.randn(4, requires_grad=True)
y = torch.randn(4, requires_grad=True)
z = x * y + 10
p = z.sum()
p.backward()
print('x:', x.data)
print('y:', y.data)
print('dp / dx:', x.grad)
print('dp / dy:', y.grad)
```

```
x: tensor([-1.3333, -0.5294, -0.0482, -0.5393])
y: tensor([ 1.9426,  1.1428, -1.8924,  0.5577])
dp / dx: tensor([ 1.9426,  1.1428, -1.8924,  0.5577])
dp / dy: tensor([-1.3333, -0.5294, -0.0482, -0.5393])
```

Automatic differentiation

```
x = torch.randn(4, requires_grad=True)
y0 = torch.randn(4, requires_grad=True)
y = y0.detach()
z = x * y + 10
p = z.sum()
p.backward()
print('x:', x.data)
print('y:', y.data)
print('dp / dx:', x.grad)
print('dp / dy:', y.grad)
```

```
x: tensor([ 2.3568,  1.1385, -0.8198,  0.5400])
y: tensor([ 1.0990, -4.3158,  2.2558,  1.3052])
dp / dx: tensor([ 1.0990, -4.3158,  2.2558,  1.3052])
dp / dy: None
```

```
x = torch.randn(3, requires_grad=True)
y = x.clone()
y.requires_grad
```

True

```
y = x.detach()
y.requires_grad
```

False

```
x = torch.randn(4, requires_grad=True)
y = torch.randn(4, requires_grad=True)
z = x + y.clone().exp_() # ошибки нет
z = x + y.exp_() # ошибка есть
```

Gradient accumulation

```
x = torch.tensor([1., 1, 1, 1], requires_grad=True)
y = x ** 2
p = y.sum()
p.backward()
print('x:', x)
print('dp / dx:', x.grad)
y = 1 / x
p = y.sum()
p.backward()
print('x:', x)
print('dp / dx:', x.grad)
```

```
x: tensor([1., 1., 1., 1.], requires_grad=True)
dp / dx: tensor([2., 2., 2., 2.])
x: tensor([1., 1., 1., 1.], requires_grad=True)
dp / dx: tensor([1., 1., 1., 1.])
```

```
x = torch.tensor([1, 1, 1, 1.], requires_grad=True)
y = x ** 2
p = y.sum()
p.backward()
print('x:', x)
print('dp / dx:', x.grad)
x.grad.detach_()
x.grad.zero_()
y = 1 / x
p = y.sum()
p.backward()
print('x:', x)
print('dp / dx:', x.grad)
```

```
x: tensor([1., 1., 1., 1.], requires_grad=True)
dp / dx: tensor([2., 2., 2., 2.])
x: tensor([1., 1., 1., 1.], requires_grad=True)
dp / dx: tensor([-1., -1., -1., -1.])
```

Leaf vs non-leaf variable

```
x = torch.randn(4, requires_grad=True)  # Leaf variable
y = x + 1                                # not a leaf variable
p = y.sum()                              # not a leaf variable
p.backward()
print('x:', x)
print('y:', y)
print('p:', p)
print('x.grad:', x.grad)
print('y.grad:', y.grad)
print('p.grad:', p.grad)
```

```
x: tensor([-1.5496,  0.3766, -0.9040, -0.2435], requires_grad=True)
y: tensor([-0.5496,  1.3766,  0.0960,  0.7565], grad_fn=<AddBackward0>)
p: tensor(1.6795, grad_fn=<SumBackward0>)
x.grad: tensor([1., 1., 1., 1.])
y.grad: None
p.grad: None
```

Leaf vs non-leaf variable

```
x = torch.randn(4, requires_grad=True)    # leaf variable
z = torch.randn(4, requires_grad=True)    # leaf variable
w = z * 2                                # not a leaf variable
y = x * w + 1    # forward pass before retaining gradient is ok
p = y.sum()
```

```
w.retain_grad()
```

```
p.backward()
print('dp / dx:', x.grad)
print('dp / dz:', z.grad)
print('dp / dw:', w.grad)
```

```
dp / dx: tensor([ 1.7251,  0.8019, -2.9055,  4.9973])
dp / dz: tensor([-3.5128,  0.0063,  1.9552, -1.8944])
dp / dw: tensor([-1.7564,  0.0031,  0.9776, -0.9472])
```

Context managers

```
x = torch.rand(3, requires_grad=True)

with torch.no_grad():
    y = x + x

y.requires_grad
```

False

```
@torch.no_grad()
def foo(x):
    return x + 2
```

```
y = foo(x)
y.requires_grad
```

False

Data loading

```
def get_loader(X, y, batch_size=64):
    train = torch.utils.data.TensorDataset(torch.from_numpy(X).float(),
                                            torch.from_numpy(y).long())
    train_loader = torch.utils.data.DataLoader(train,
                                                batch_size=batch_size)
    return train_loader

# for final model:
train_loader_full = get_loader(X_train, y_train)
test_loader = get_loader(X_test, y_test)
# for validation purposes:
train_loader = get_loader(X_train[:15000], y_train[:15000])
val_loader = get_loader(X_train[15000:30000], y_train[15000:30000])
```

Creation of a neural network

```
class CNN(nn.Module):
    def __init__(self, k=1):
        super(CNN, self).__init__()
        self._layers = nn.Sequential(
            nn.Conv2d(1, k * 6, kernel_size=5),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2),
            nn.Conv2d(k * 6, k * 16, kernel_size=5),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2),
            nn.Flatten(),
            nn.Linear(256 * k, 84),
            nn.ReLU(),
            nn.Linear(84, 10)
        )

    def forward(self, x):
        return self._layers(x)
```

```
cnn = CNN()
```

```
def count_parameters(model):
    return sum(param.data.numpy().size for param \
                in model.parameters() if param.requires_grad)
```

```
count_parameters(cnn)
```

```
25010
```

```
optimizer = torch.optim.Adam(cnn.parameters(), lr=1e-3)
```

Neural network training

```
def train_epoch(model, optimizer, train_loader, criterion, device):  
  
    model.to(device)  
    model.train()  
    for x, y in train_loader:  
        optimizer.zero_grad()  
        loss = criterion(model(x.to(device)), y.to(device))  
        loss.backward()  
        optimizer.step()
```

Neural network testing

```
def evaluate_loss_acc(loader, model, criterion, device):  
  
    model.eval()  
    model.to(device)  
    total_loss = 0.0  
    n_correct = 0  
    n_total = 0  
    with torch.no_grad():  
        for x, y in loader:  
            x = x.to(device)  
            y = y.to(device)  
            out = model(x)  
            total_loss += criterion(out, y).item()  
            n_total += out.shape[0]  
            n_correct += torch.sum(y == out.argmax(dim=1)).item()  
    return total_loss, n_correct / n_total
```